



Satoripop

Shine clever !

Boussadia Chaabane



Routing?

Services?

TS



component?



DataBinding?

I'll present you

Angular (Not AngularJS)

Why Angular ?

Data Binding

- Move my JS code to the view without writing any code !

Custom Components

- Build your own components
- reusable pieces
- played well with web component



Dependency Injection

- Writing modular services
- Have them injected wherever they are needed
- improving testing

and more . . .

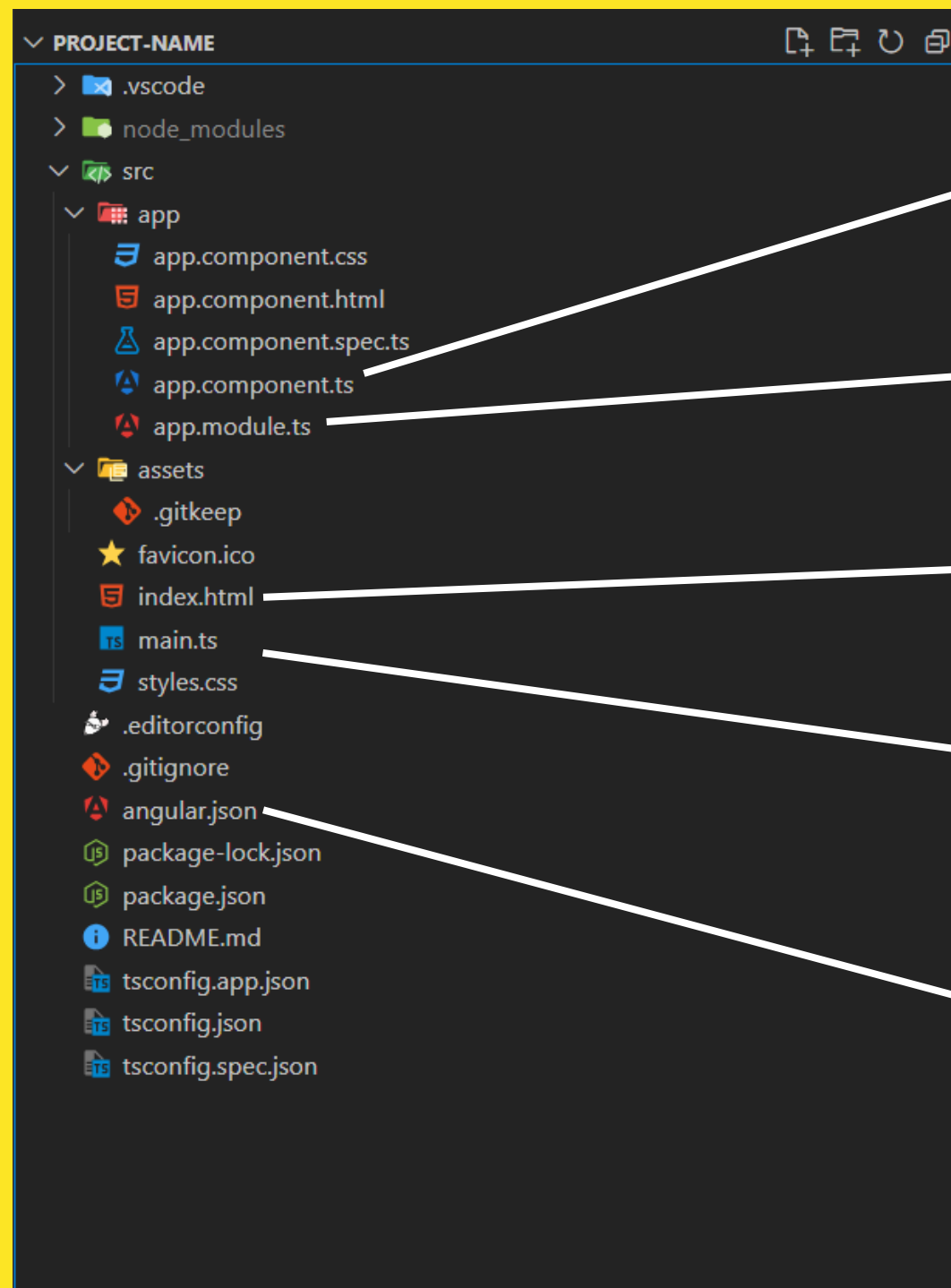


Angular Basics

Creating a new application is as simple by running the following command :

ng new project-name

When you run this command , It will automatically a skeleton application under the folder
project-name



The root component

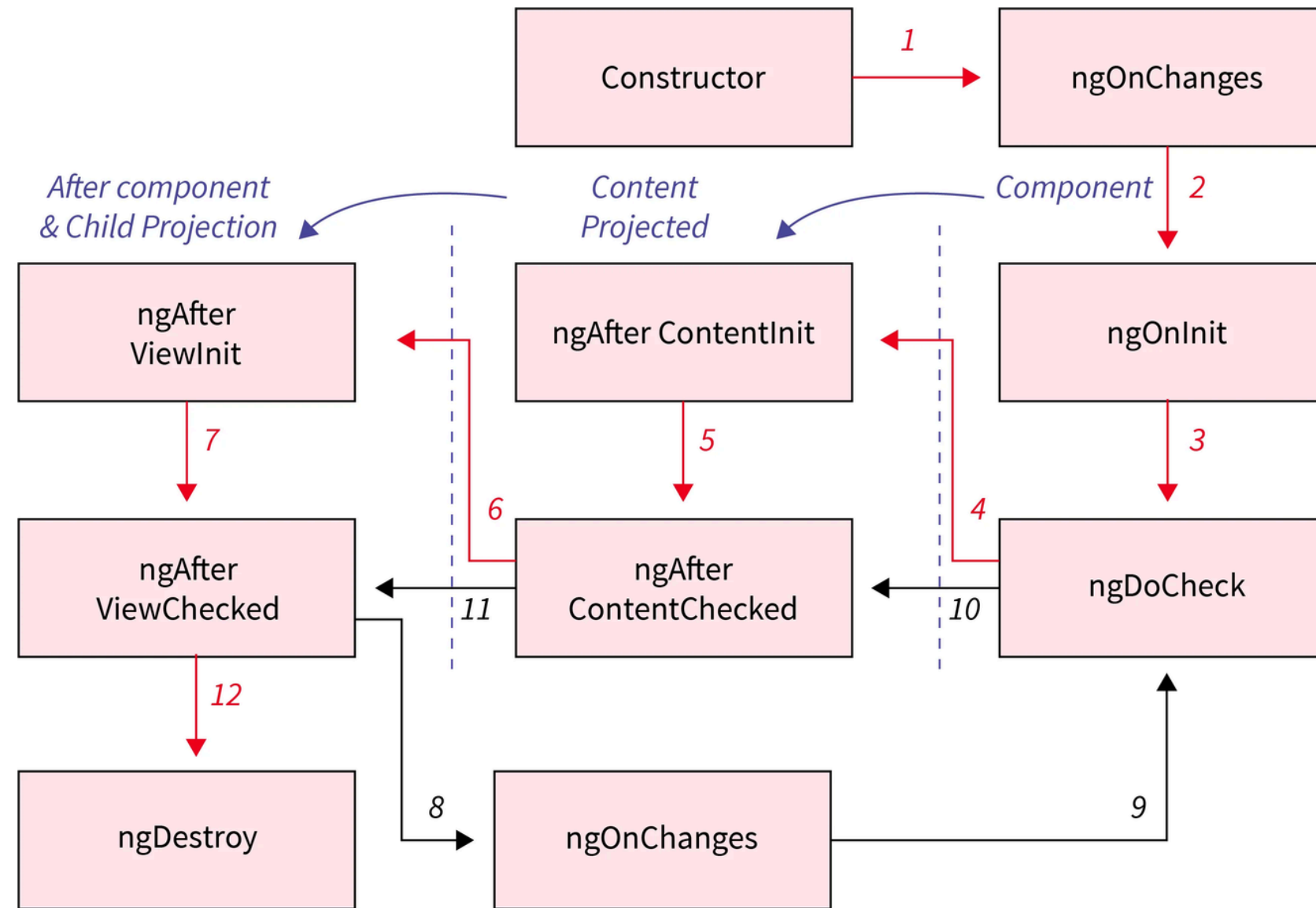
The core configuration of the application

here combines all vendor libraries , templates , ... and injects them in this file (webpack)

It identifies which angular module will be loaded

stores informations about the project's architecture , dependencies , build and test configurations and more

Life cycle



Red - only first time

Black - In every change detection



Components

Components

To create a component : **ng g c component-name**
If manually , YOU need to update app.module.ts

In **TS** file :

Selector :

In an other html file , we need to use

<selector-name></selector-name>

so it will load the html , styling and the TS code of this componeny

Styles



Template



Inputs

As a hint : passing the data from father component to child component

```
@Input() public stock: Stock;
```

It automatically allows you to pass in values to the component for that particular input

```
<app-stock-item [stock]="stockObj"></app-stock-item>
```

This some data initialized in the component



Output

```
<button (click)="onToggleFavorite($event)"
  *ngIf="!stock.favorite">Add to Favorite</button>
```

Component A

```
@Output() private toggleFavorite: EventEmitter<Stock>;
```

```
onToggleFavorite(event) {
  this.toggleFavorite.emit(this.stock);
}
```

Component B

```
<app-stock-item [stock]="stock"
  (toggleFavorite)="onToggleFavorite($event)">
</app-stock-item>
```

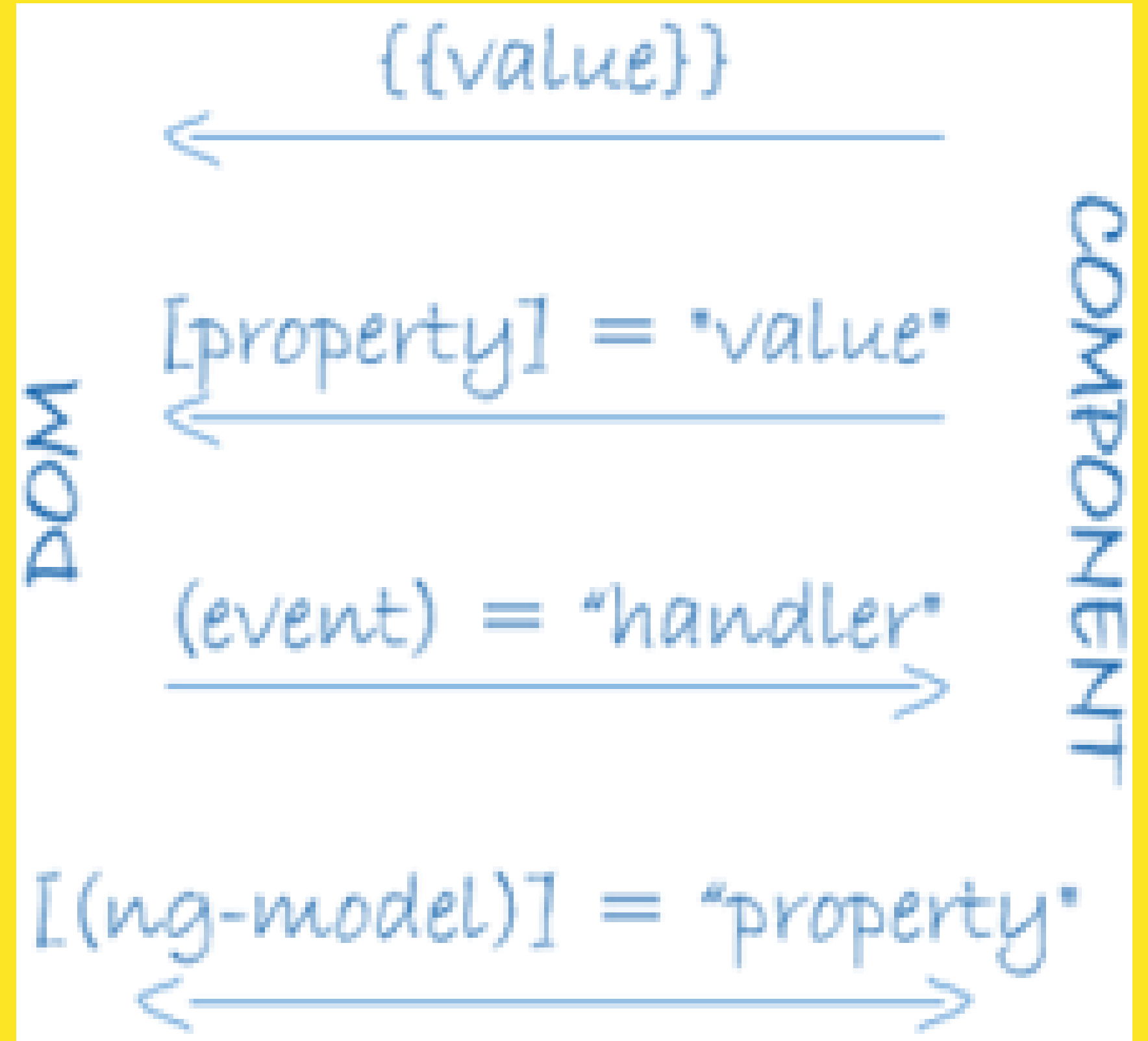
```
onToggleFavorite(stock: Stock) {
  console.log('Favorite for stock ', stock, ' was triggered');
  this.stock.favorite = !this.stock.favorite;
}
```

When I click to the button , the method will emit the data to the other component who is about to receive it

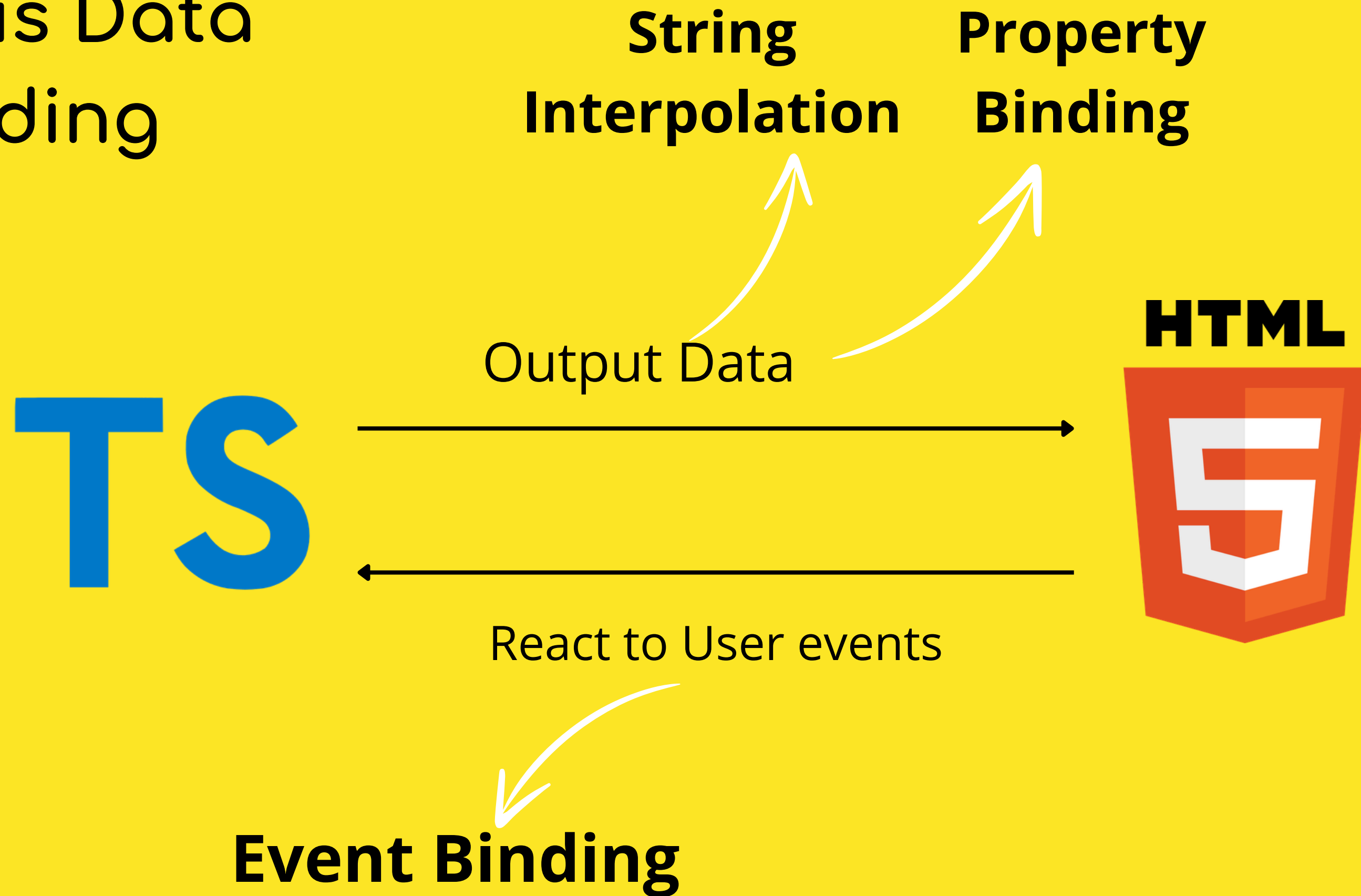
The Other component will receive the data and will use it



Data Binding



What is Data Binding



We can mention also the Two Way Binding



Directives



Directives

Structural directives

add/remove elements

***NgIf**

Allows you to conditionally hide or show elements in UI

***ngFor**

It is used for creating multiple elements , usually one for each instance of an object

Attribute directives

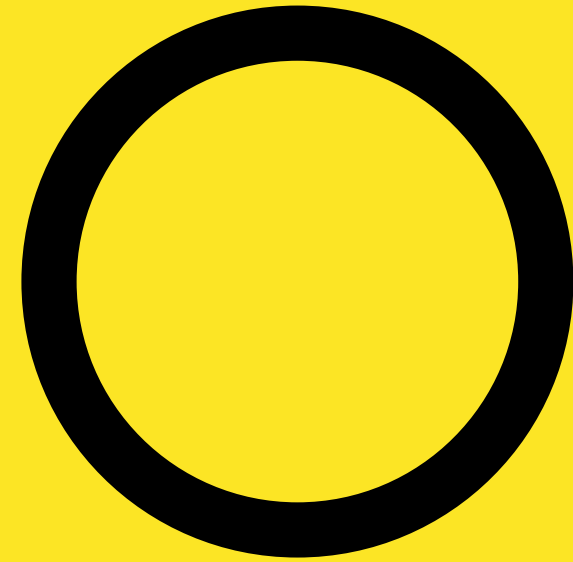
change the behaviour of an element

NgStyle

like ngClass but only with CSS properties

NgClass

apply or remove CSS classes simultaneously from an element in our html



Services

provide a way for you to separate Angular app data and functions that can be used by multiple components in your app.

Component A

sender

call a method that emit
event and pass data

```
export class SenderComponent {  
  newData: string = '';  
  
  constructor(private dataService: DataService) {}  
  
  sendData() {  
    this.dataService.changeData(this.newData);  
  }  
}
```

Angular Service
+Id parameter
+onDataRecieverEvent
+emitEvent(id) Method

```
@Injectable({  
  providedIn: 'root'  
})  
export class DataService {  
  dataChanged: EventEmitter<string> = new EventEmitter<string>();  
  
  constructor() { }  
  
  changeData(data: string) {  
    this.dataChanged.emit(data);  
  }  
}
```

subscribe to the event

Component B

reciever

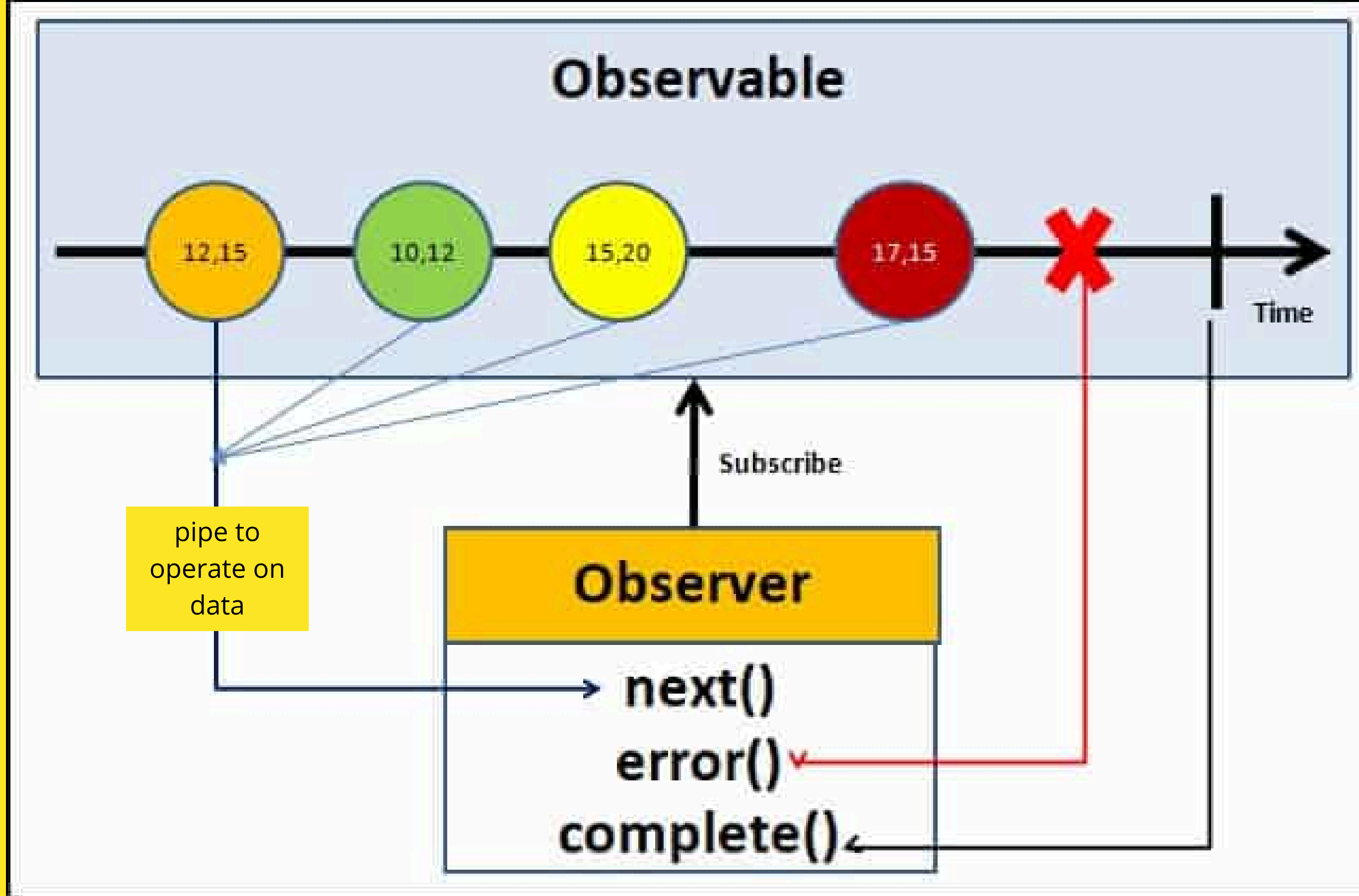
Notify the data recieved from
sender to reciever component

```
export class ReceiverComponent implements OnInit {  
  receivedData: string;  
  
  constructor(private dataService: DataService) {}  
  
  ngOnInit() {  
    this.dataService.currentData.subscribe(data => this.receivedData = data);  
  }  
}
```



Observables

What is Observable





Routing

```
<router-outlet></router-outlet>
```

```

18  const routes : Routes = [
19      /** The home route */
20      { path: '', component: HomeComponent },
21      /** The users route
22       * @children [UserComponent]
23       */
24      { path: 'users', component: UsersComponent , children: [
25          { path: ':id/:name' , component: UserComponent},
26      ]},
27      /** The servers route
28       * @ canActivateChild [AuthGuard]
29       * @children [ServerComponent, EditServerComponent]
30       */
31      { path: 'servers',
32        // canActivate: [AuthGuard],
33        canActivateChild: [AuthGuard],
34        component: ServersComponent,
35
36        children: [
37            { path: ':id' ,resolve: {server: serverResolver}, component: ServerComponent},
38            { path: ':id/edit', component: EditServerComponent, canActivate: [CanDeactivateGuard]},
39        ]
40      },
41      /** The error page route */
42      {
43          path: 'not-found',
44          component: ErrorPageComponent,
45          data: {message: 'Error : Page not found'}
46      },
47      /** Redirect all other paths to the not-found page */
48      { path: '**', redirectTo: '/not-found'}
49  ];
50

```

Routing configuration

how to navigate ?

```
<li role="presentation"  
  routerLinkActive="active"  
><a routerLink="/servers">Servers</a></li>
```

Using RouterLink

```
constructor(private serversService: ServersService , private route : ActivatedRoute, private router : Router) { }  
/**  
 * Navigates to the edit route for the current server  
 */  
onEdit() {  
  /**  
   * This will navigate to the edit route while preserving all query parameters and the fragment  
   */  
  this.router.navigate(['edit'],{relativeTo: this.route , queryParamsHandling: 'preserve'});  
}
```

Navigating progmatically

Instead of using href

Route Guards

loading or unloading of a route based on your own conditions

Step 1 : Create a service for authentication

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { UserStoreService } from '../services/user-store.service';
import { Observable } from 'rxjs/Observable';

@Injectable()
export class AuthGuard implements CanActivate {

  constructor(private userStore: UserStoreService,
    private router: Router) {}

  canActivate(): boolean {
    console.log('AuthGuard#canActivate called');

    if (this.userStore.isLoggedIn()) { return true };

    console.log('AuthGuard#canActivate not authorized to access page');
    // Can store current route and redirect back to it
    // Store it in a service, add it to a query param
    this.router.navigate(['login']);

    return false;
  }
}
```

Step 2 : Create a guard
implementing
CanActivate

```
/** Imports unchanged, skipping for brevity */
import { AuthGuard } from '../guards/auth.guard';

const appRoutes: Routes = [
  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  { path: 'register', component: RegisterComponent },
  { path: 'stocks/list', component: StockListComponent,
    canActivate: [AuthGuard] },
  { path: 'stocks/create', component: CreateStockComponent,
    canActivate: [AuthGuard] },
  { path: 'stock/:code', component: StockDetailsComponent,
    canActivate: [AuthGuard] },
  { path: '**', redirectTo: '/register' }
];

/** Code unchanged, skipping for brevity */
export class AppRoutesModule { }
```

Update **app-routes.module.ts**

Preloading data using resolve

The main idea is :

```
@Injectable()
export class StockLoadResolverService implements Resolve<Stock> {

  constructor(private stockService: StockService) { }

  resolve(route: ActivatedRouteSnapshot,
          state: RouterStateSnapshot):
    Stock | Observable<Stock> | Promise<Stock> {
    const stockCode = route.paramMap.get('code');
    return this.stockService.getStock(stockCode);
  }
}
```

We first implement the Resolve method in our service file

and then we will preload the data in the component , don't forget to use subscription in the component to keep it up to date

```
{ path: 'stock/:code', component: StockDetailsComponent,
  canActivate: [AuthGuardService],
  resolve: { stock: StockLoadResolverService } },
```



Shine clever !

Thanks for watching